

LAB NOTES · ISSUE 03 OF 04

Lab: MLOps on Kubernetes with GitOps.

Train, gate, promote and serve — then deploy it the GitOps way.

03

AR

Akhilesh Ranjan Singh

Senior DevSecOps Engineer · github.com/iarsingh

Swipe →

Training lab

- 1 train.py
- 2 Register version
- 3 evaluate.py
- 4 Gate and promote
- 5 Inference service reloads

Promotion happens only if the evaluation gate passes.

Inference service

01 **/health and /ready**

Readiness reflects whether a model is loaded.

02 **/metrics**

Prometheus metrics for latency and requests.

03 **/reload**

Pick up a newly promoted version.

04 **Registry with local fallback**

The loader can serve from a local copy.

GitOps lab

- 1 CI builds image
- 2 promote-image.sh
- 3 Argo CD syncs
- 4 drift-check.sh
- 5 rollback.sh

Promotion and rollback are scripted Git changes, not kubectl commands.



What both labs share

- ☒ Health and readiness endpoints

- ☒ Prometheus metrics

- ☒ CI workflow with tests

- ☒ Terraform for registry and storage

- ☒ A rehearsed rollback path

WHY TWO LABS

Model lifecycle and deploy lifecycle are different loops.

Keeping them separate made each one easier to reason about and test.



KEY TAKEAWAY

Gate the model, then let GitOps handle the deploy.

- Evaluation gate before promotion
- Readiness reflects model state
- Scripted promote, drift check, rollback

YOUR TURN

Do you keep model promotion and service deployment in one pipeline or two?

FOLLOW AKHILESH RANJAN SINGH

Next in the series: #04 Lab: synthetic datasets for MLOps and GenAI practice

Save · Repost

